Quicksort performance

Luis Martin Pena

2026-11-04

Setup

- Machine: Intel Core Ultra 9 285H.
- 16 physical cores, 16 logical CPUs.
- Three implementations: sequential, parallel and libc.
- Each implementation sorts a copy of the same array.
- Order: sequential, parallel, libc.
- Compiled with `-O0`.

First experiment

I started with five array sizes, from 10,000 to 10 million elements.

- Ten runs per size.
- Runs mixed across sizes.
- Times in seconds.

```
library(ggplot2)

data <- read.csv(
  "data/aptikal-trpl_2026-10-27/measurements_simple_140210.csv"
)

table(data$size)
```

```
##
## 10000 1e+05 1e+06 5e+06 1e+07
##    10    10    10    10    10
```

Execution time

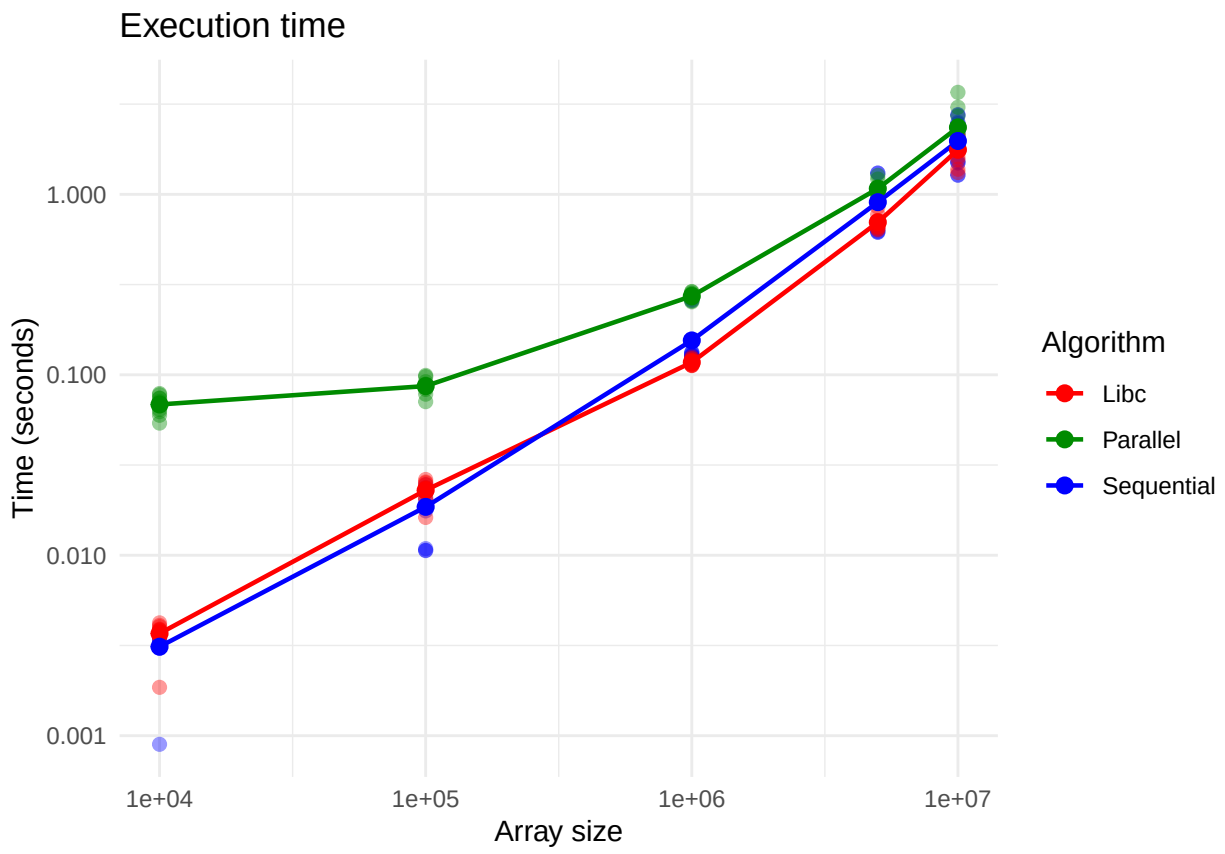
Let's first plot the execution times. Points are individual runs; lines connect the arithmetic means. Both axes are logarithmic.

```
times <- rbind(
  data.frame(
    size = data$size,
    algorithm = "Sequential",
    time = data$sequential
```

```
  ),
  data.frame(
    size = data$size,
    algorithm = "Parallel",
    time = data$parallel
  ),
  data.frame(
    size = data$size,
    algorithm = "Libc",
    time = data$builtin
  )
)

mean_times <- aggregate(time ~ size + algorithm, data = times, mean)

ggplot(times, aes(x = size, y = time, colour = algorithm)) +
  geom_point(alpha = 0.4, size = 2) +
  geom_line(data = mean_times, linewidth = 0.8) +
  geom_point(data = mean_times, size = 2.5) +
  scale_colour_manual(
    values = c(
      "Sequential" = "blue",
      "Parallel" = "green4",
      "Libc" = "red"
    )
  ) +
  scale_x_log10() +
  scale_y_log10() +
  labs(
    title = "Execution time",
    x = "Array size",
    y = "Time (seconds)",
    colour = "Algorithm"
  ) +
  theme_minimal()
```



What do we see:

- Parallel is much slower for small arrays.
- The relative difference gets smaller as array size increases.
- Sequential and libc have similar times.
- Results vary between runs.

Parallel speedup

Let's compare parallel with sequential for each run:

Speedup = sequential time / parallel time

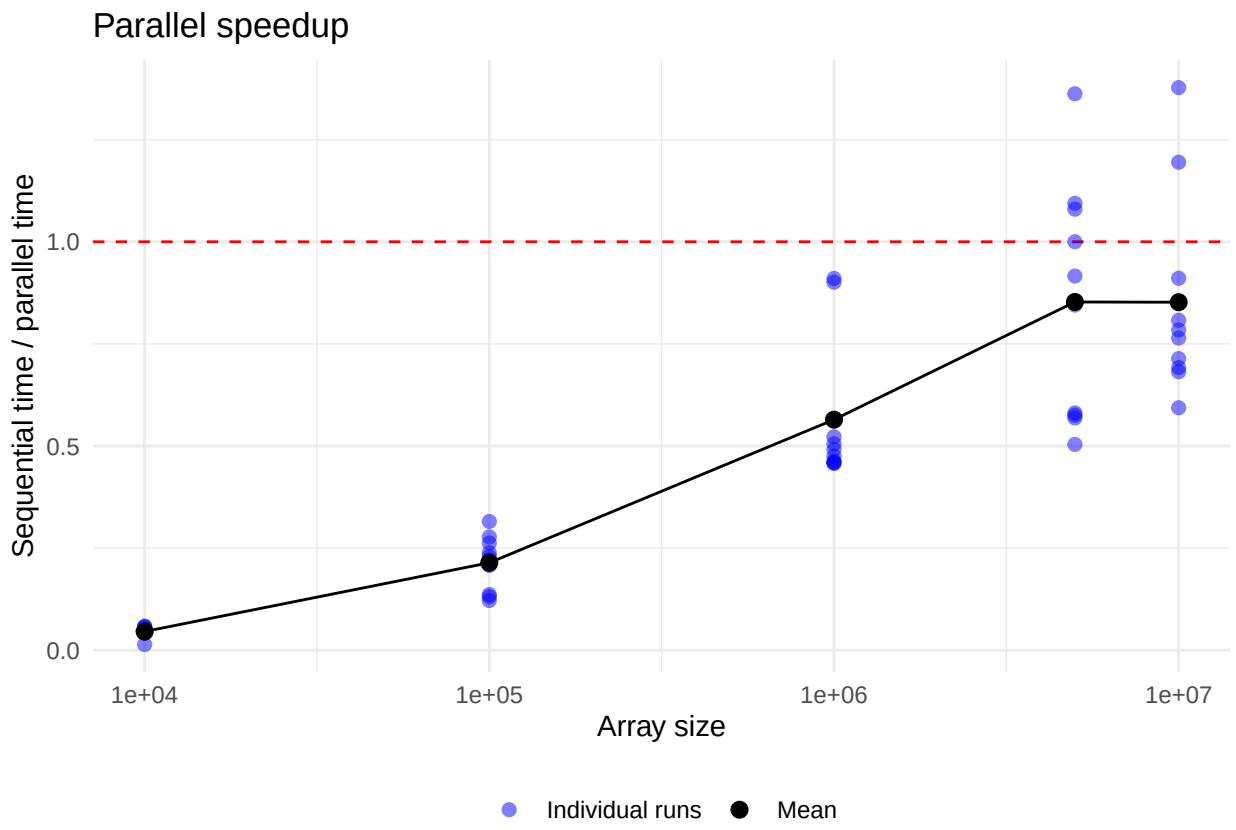
A value above 1 means parallel is faster; below 1 means it is slower. I'll try both a dot plot and a boxplot (but for the boxplot it willbe interesting to keep the dots anyways, let's do a transparent box)

```
data$speedup <- data$sequential / data$parallel

mean_speedup <- aggregate(speedup ~ size, data = data, mean)

# Dot plot
ggplot(data, aes(x = size, y = speedup)) +
  geom_hline(
    yintercept = 1,
    linetype = "dashed",
    colour = "red"
```

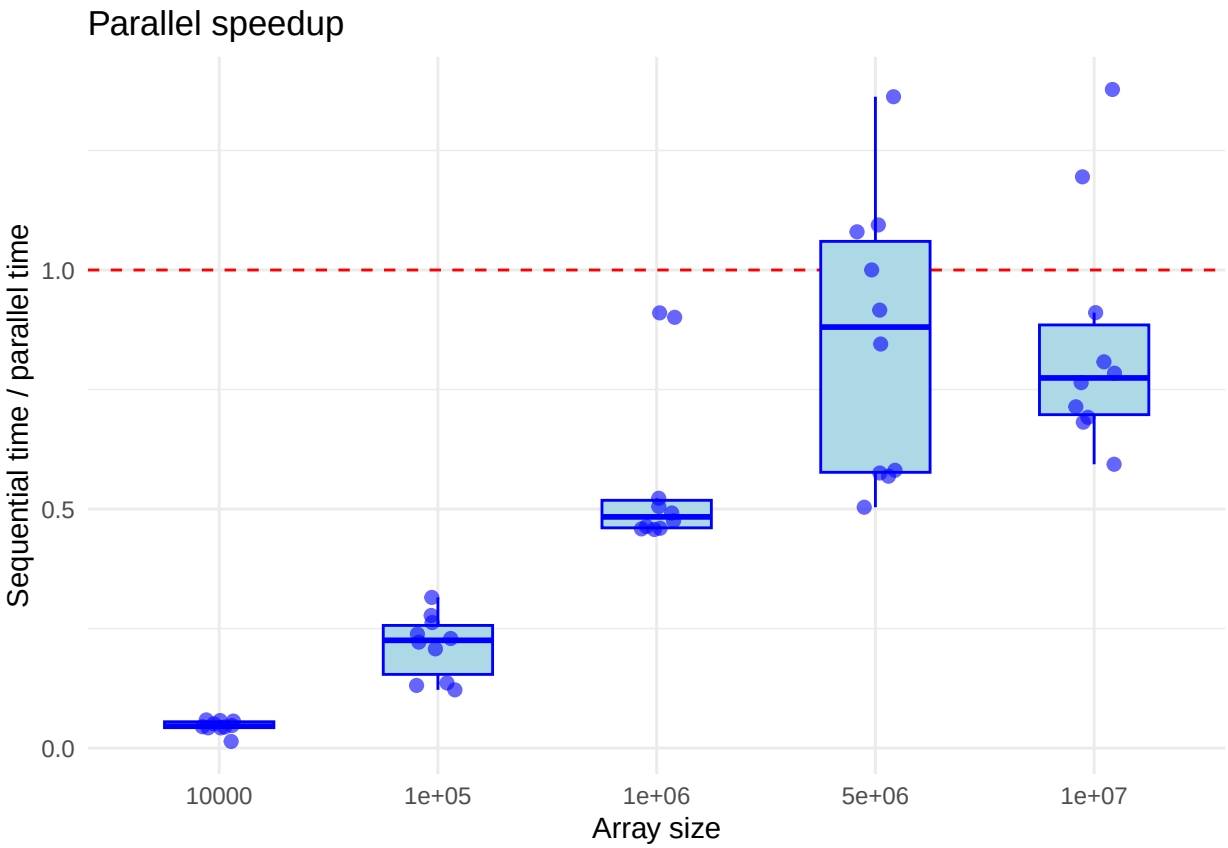
```
  ) +
  geom_point(
    aes(colour = "Individual runs"),
    alpha = 0.5,
    size = 2
  ) +
  geom_line(
    data = mean_speedup,
    aes(group = 1),
    colour = "black"
  ) +
  geom_point(
    data = mean_speedup,
    aes(colour = "Mean"),
    size = 2.5
  ) +
  scale_colour_manual(
    name = NULL,
    values = c("Individual runs" = "blue", "Mean" = "black")
  ) +
  scale_x_log10() +
  labs(
    title = "Parallel speedup",
    x = "Array size",
    y = "Sequential time / parallel time"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```




```r
data$speedup <- data$sequential / data$parallel

# Boxplot
ggplot(data, aes(x = factor(size), y = speedup)) +
  geom_hline(
    yintercept = 1,
    linetype = "dashed",
    colour = "red"
  ) +
  geom_boxplot(
    fill = "lightblue",
    colour = "blue",
    outlier.shape = NA,
    width = 0.5
  ) +
  geom_jitter(
    colour = "blue",
    alpha = 0.6,
    width = 0.1,
    size = 2
  ) +
  labs(
    title = "Parallel speedup",
    x = "Array size",
    y = "Sequential time / parallel time"
  ) +
```

```
theme_minimal()
```



- Mean speedup stays below 1 for all tested sizes.
- Parallel is faster in some runs at 5 and 10 million.
- The mean is almost the same at 5 and 10 million.

Parallel seems to catch up as the array gets bigger, but 10 runs per size don't tell us much about how stable this result is. Let's look back at the original experiment before running more tests.

Notes on the original experiment

- Five sizes, with five runs per size.
- Nothing tested between 100,000 and 1 million.
- The crossover at 400,000 was not directly measured.
- The random seed comes from the current time in seconds.
- Runs started in the same second may use the same input.
- Results come from one machine.

Let's try more sizes and more runs on my machine. I'll also go beyond 10 million elements and mix the order of the runs again.

Extended experiment

- Seven sizes: 1 million to 20 million.

- Twenty runs per size.
- Same implementations and compilation settings.

```
lines <- readLines("data/aptikal-trpl_2026-11-10/measurements_extended_161703.txt")

blocks <- split(lines, rep(seq_len(length(lines) / 5), each = 5))

extended <- do.call(rbind, lapply(seq_along(blocks), function(i) {
  block <- blocks[[i]]

  data.frame(
    run_order = i,
    size = as.numeric(sub("Size: ", "", block[1])),
    repetition = as.numeric(sub("Repetition: ", "", block[2])),
    sequential = as.numeric(sub("Sequential quicksort took: (.*) sec\\.", "\\1", block[3])),
    parallel = as.numeric(sub("Parallel quicksort took: (.*) sec\\.", "\\1", block[4])),
    builtin = as.numeric(sub("Built-in quicksort took: (.*) sec\\.", "\\1", block[5]))
  )
}))

table(extended$size)
```

```
##
##   1e+06 2500000   5e+06 7500000   1e+07 1.5e+07   2e+07
##      20      20      20      20      20      20      20
```

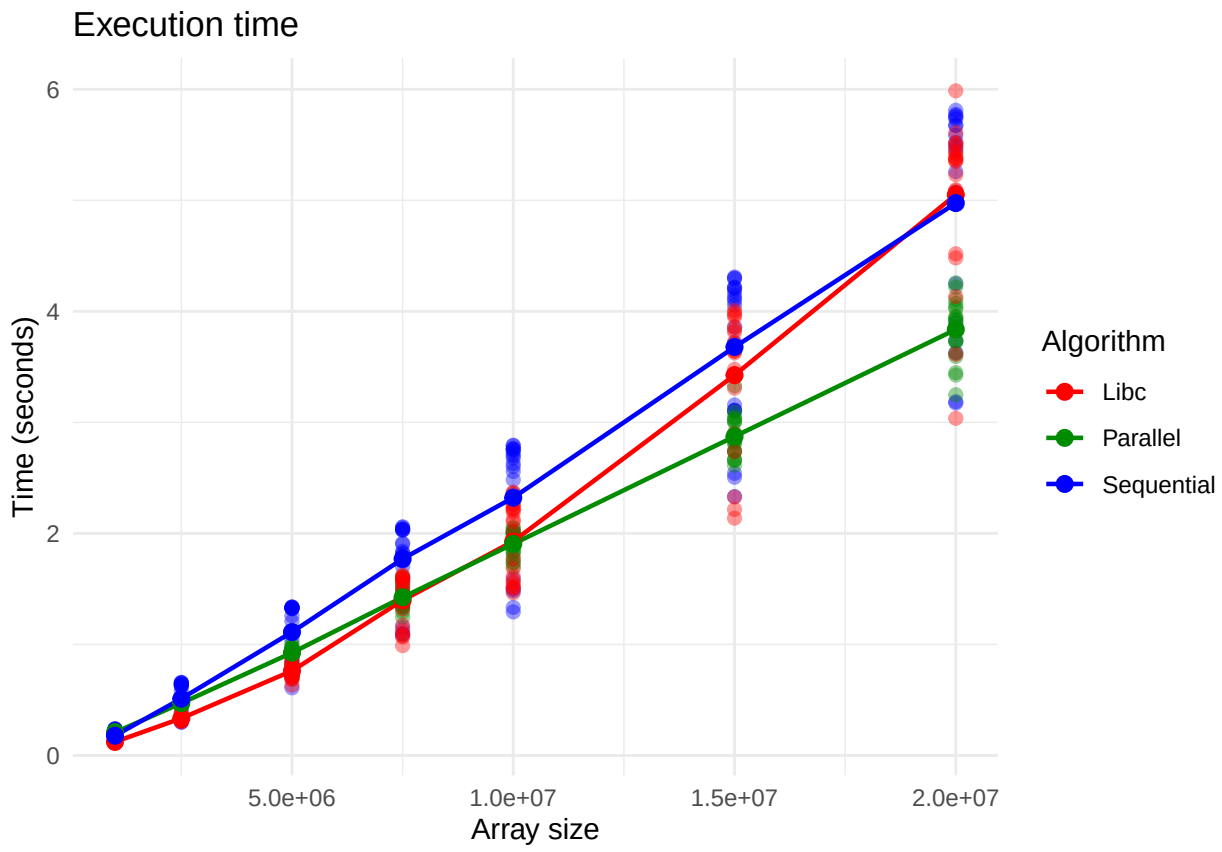
Execution time

Same plot as before: individual runs and mean times.

```
extended_times <- rbind(
  data.frame(
    size = extended$size,
    algorithm = "Sequential",
    time = extended$sequential
  ),
  data.frame(
    size = extended$size,
    algorithm = "Parallel",
    time = extended$parallel
  ),
  data.frame(
    size = extended$size,
    algorithm = "Libc",
    time = extended$builtin
  )
)

extended_means <- aggregate(
  time ~ size + algorithm,
  data = extended_times,
  FUN = mean
)
```

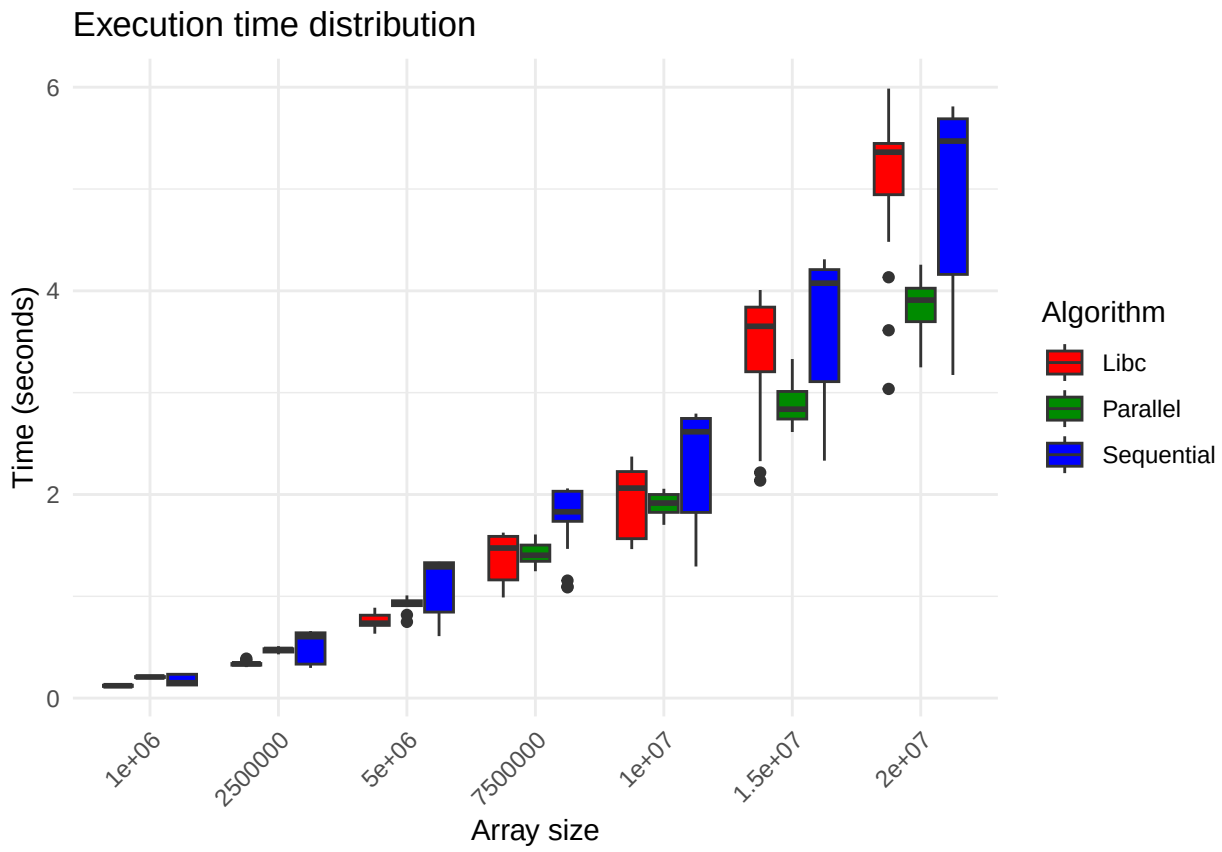
```
ggplot(extended_times, aes(x = size, y = time, colour = algorithm)) +
  geom_point(alpha = 0.4, size = 2) +
  geom_line(data = extended_means, linewidth = 0.8) +
  geom_point(data = extended_means, size = 2.5) +
  scale_colour_manual(
    values = c(
      "Sequential" = "blue",
      "Parallel" = "green4",
      "Libc" = "red"
    )
  ) +
  labs(
    title = "Execution time",
    x = "Array size",
    y = "Time (seconds)",
    colour = "Algorithm"
  ) +
  theme_minimal()
```



The points overlap quite a bit. Let's try boxplots instead.

```
ggplot(
  extended_times,
  aes(x = factor(size), y = time, fill = algorithm)
) +
```

```
  geom_boxplot() +
  scale_fill_manual(
    values = c(
      "Sequential" = "blue",
      "Parallel" = "green4",
      "Libc" = "red"
    )
  ) +
  labs(
    title = "Execution time distribution",
    x = "Array size",
    y = "Time (seconds)",
    fill = "Algorithm"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



That's a lot of boxes, but the distributions are easier to see.

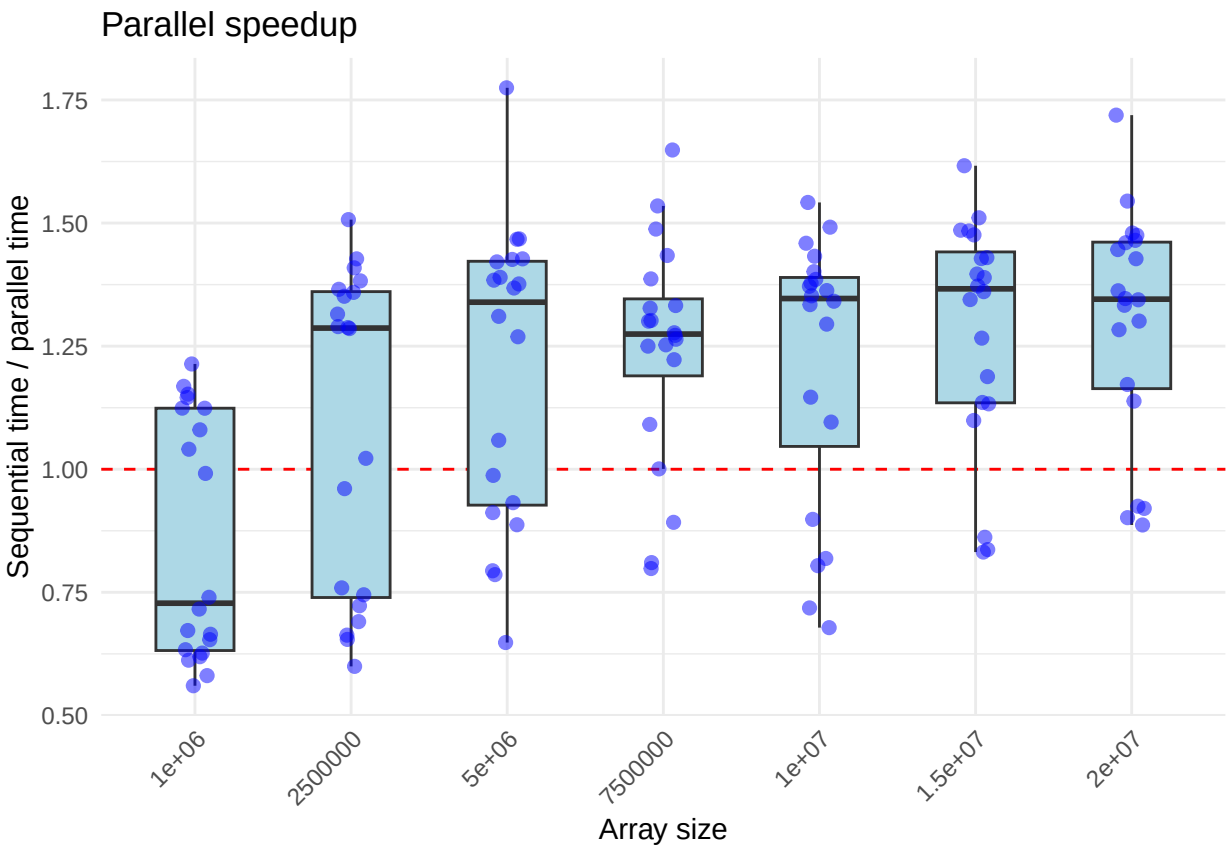
- libc has the lowest median time at 1 million.
- Parallel and libc have close median times at 10 million.
- Parallel has the lowest median time at 15 and 20 million.
- Sequential and libc show wider distributions at larger sizes.
- The distributions overlap: the fastest algorithm can change between runs.

Parallel speedup

Let's look at the speedup again. I'll keep the boxplot this time.

```
extended$speedup <- extended$sequential / extended$parallel

ggplot(extended, aes(x = factor(size), y = speedup)) +
  geom_hline(yintercept = 1, linetype = "dashed", colour = "red") +
  geom_boxplot(fill = "lightblue", outlier.shape = NA, width = 0.5) +
  geom_jitter(colour = "blue", alpha = 0.5, width = 0.1, size = 2) +
  labs(
    title = "Parallel speedup",
    x = "Array size",
    y = "Sequential time / parallel time"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



- At 1 million, the median speedup is below 1.
- From 2.5 million onward, the median is above 1.
- Parallel is still slower in some individual runs at every size.